InstaBuy- e-Commerce Website

1. Executive Summary

InstaBuy is a distributed e-commerce platform designed to deliver scalability, resilience, and high availability. The system adopts a microservices architecture, where core business capabilities—including Authentication, Catalog Management, Ordering, Inventory, and Payments—are decoupled into independent services. This separation ensures that failures or performance issues in one domain do not cascade across the entire platform, thereby improving fault tolerance and operational stability.

The backend is implemented using **Spring Boot**, enabling rapid development, ease of deployment, and seamless integration of microservices. The frontend is built with **React**, providing a responsive and intuitive user experience across devices.

To ensure reliable data consistency and coordination across distributed services, InstaBuy leverages the **Saga choreography (event-driven) pattern** for managing long-running transactions. **ActiveMQ** is used as the messaging broker to facilitate **asynchronous, event-based communication** between microservices. This approach allows services to react to events independently, reduces tight coupling, and enhances system scalability while maintaining eventual consistency across the platform.

Together, these architectural choices position InstaBuy as a robust, scalable, and modern e-commerce solution capable of supporting high traffic and evolving business requirements.

2. Detailed Tech Stack

2.1 Frontend

React.js and Bootstrap 5: Used to build a fast, component-based user interface with efficient state management. Enhances performance through virtual DOM rendering and reusable UI components.
Axios: Handles secure and efficient HTTP communication between frontend and backend services. Supports interceptors for JWT authentication and centralized error handling.
React Router: Enables client-side routing for seamless navigation without full page reloads. Supports protected and dynamic routes to improve user experience.

2.2 Backend

Spring Boot: Simplifies microservice development with auto-configuration and production-ready features. Enables rapid development of REST APIs with minimal boilerplate code.

Spring Cloud Gateway: Acts as a single-entry point for routing, authentication, and load balancing. Improves security and decouples clients from internal microservices.
Netflix Eureka: Provides dynamic service discovery for registering and locating microservices. Enhances scalability and fault tolerance without hard-coded service endpoints.

2.3 Databases

SQL (MySQL): Used for transactional data requiring strong consistency and ACID compliance. Ensure reliable handling of users, orders, and inventory data.
NoSQL (MongoDB): Supports flexible, schema-less storage for diverse product structures. Allows fast iteration and scalability for frequently changing product data.

2.4 Messaging

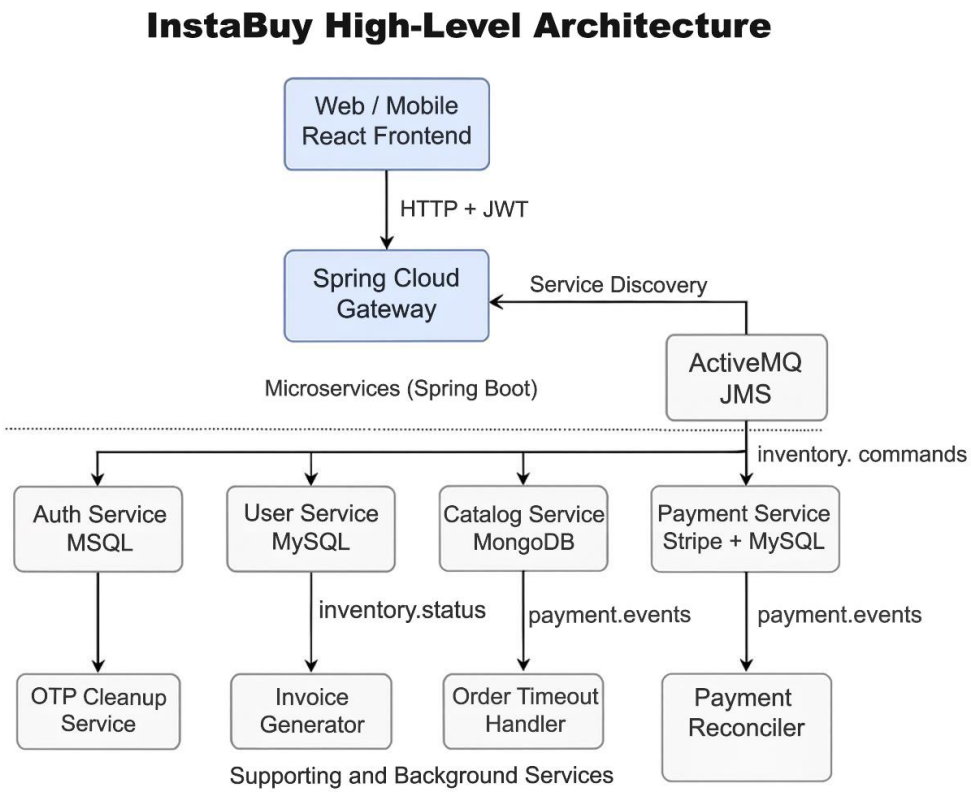
Apache ActiveMQ (JMS): ActiveMQ enables asynchronous communication between microservices using message queues and event topics. It is a core component in implementing the Saga event-driven pattern, allowing services such as Orders, Inventory, and Payments to communicate without tight coupling. This improves resilience, ensures eventual consistency, and enables rollback or compensation actions in case of transaction failures.

2.5 Security

JJWT (JSON Web Token): Provides stateless authentication for secure communication between services. Eliminates server-side session management and improves scalability.
BCrypt Password Hashing: Used to securely hash and store user passwords. Protects against brute-force and rainbow table attacks.

2.6 Payments

Stripe API: Handles secure payment processing and PCI-DSS compliance.
Supports **real-time transaction validation** and webhook-based event handling.



3. Service Discovery (Netflix Eureka):

Netflix Eureka acts as the **central service registry** for the InstaBuy microservices ecosystem. In a distributed environment where services are dynamically scaled, restarted, or redeployed, IP addresses and ports frequently change, making static configuration impractical.

Each microservice registers itself with Eureka at startup and sends periodic heartbeats to indicate availability. The API Gateway and internal clients (Feign) query Eureka at runtime to locate healthy service instances. This enables **dynamic routing**, seamless horizontal scaling, and **eliminates hardcoded service URL**s, significantly improving maintainability and fault tolerance.

4. Complete API Reference (Synchronous REST APIs)

All external client requests are routed through the **API Gateway (Port 8080)**. These APIs are used when immediate responses are required.

4.1 User & Security APIs

POST /api/auth/register

Creates a new user account with passwords securely hashed using BCrypt.

POST /api/auth/login

Validates credentials and returns a signed JWT containing roles and permissions.

GET /api/users/profile/{id}
Secured endpoint to retrieve user profile data while hiding internal fields.

4.2 Catalog & Product APIs

GET /api/products
Returns a paginated list of products from MongoDB.
GET /api/products/search/{query}
Performs full-text search across product attributes.
POST /api/products (ROLE_VENDOR)
Allows vendors to create and manage their own product listings.

4.3 Order APIs

POST /api/orders/create (ROLE_USER)
Validates cart data and initiates the Saga workflow for order placement.
GET /api/orders/{id}
Fetches the details of a specific order using the order ID.
GET /api/orders/get-all-orders *(ROLE_USER)*
Retrieves all orders created by the logged-in customer, enabling order history viewing.
GET /api/sales/{id} *(ROLE_VENDOR)*
To allow a vendor to view the details of a specific sales transaction that belongs to them for verification, auditing, and transparency.
GET /api/sales/report *(ROLE_VENDOR)*
To provide vendors with a summary of their sales performance within a specified date range, enabling revenue analysis and trend tracking.
GET /api/sales/total-report *(ROLE_VENDOR)*
To provide a high-level overview of total lifetime sales for the logged-in vendor.
GET /api/sales/vendor/{id} *(ROLE_ADMIN)*
To allow administrators to monitor and analyze the total sales performance of a specific vendor for oversight and compliance.
GET /api/sales/vendor/total/{id} *(ROLE_ADMIN)*
To enable administrators to perform time-based analysis of vendor sales, supporting reporting, payouts, and business decisions.

4.4 Inventory APIs

POST /api/inventory (ROLE_VENDOR)
Allows a vendor to add a product to inventory with quantity and pricing
GET /api/inventory/{id} *(ROLE_ADMIN)*
Fetches inventory details for a specific product entry.
GET /api/inventory/vendor/{vendorId} *(ROLE_ADMIN)*
Allows administrators to view all inventory items of a vendor.
GET /api/inventory/vendor *(ROLE_VENDOR)*

Allows a vendor to view all their inventory items.
GET /api/inventory/product/{productId} *(ROLE_USER)*
Fetches all vendor inventories for a specific product, used for availability checks.
PATCH /api/inventory/product/{id}
Allows Order Service to update inventory quantity during order placement or cancellation.
PATCH /api/inventory/{id} *(ROLE_VENDOR)*
Allows vendors to update price, discount, or quantity.
DELETE /api/inventory/{id} *(ROLE_VENDOR)*
Allows vendor to remove a product from inventory

4.5 Payment APIs
POST /api/payment/create-checkout-session (ROLE_USER)
Allows a logged-in customer to initiate a Stripe checkout session for a given order.
POST /api/payment/payment/confirm *(ROLE_USER)*
Confirms the payment result from Stripe and updates the system accordingly using events.
POST /api/payment/{id}/payment/fail *(ROLE_USER)*
Allows explicit marking of order payment as failed, typically during payment cancellation or timeout.

.

5. Asynchronous Communication (ActiveMQ – Event-Driven)

InstaBuy adopts an **event-driven asynchronous communication model** using **ActiveMQ** to enable loose coupling, scalability, and resilience across its microservices architecture. Instead of synchronous service-to-service calls, services communicate by publishing and consuming events through messaging queues and topics.
This approach is a foundational enabler for implementing the **Saga pattern** to manage distributed transactions across the **Order, Inventory, and Payment** services while avoiding tight coupling, blocking operations, and complex distributed locks.

5.1 Architectural Overview

ActiveMQ acts as the **central messaging backbone** that transports domain events and commands between independently deployed services. Each service performs its **local transaction autonomously** and reacts to events published by other services, ensuring **eventual consistency** rather than immediate consistency.
The system follows a **choreography-based** Saga pattern, where:

- There is no centralized orchestrator
- Each service reacts to events and publishes the next event in the workflow
- Failures are handled via compensating actions rather than **two-phase commits**

5.2 Saga-Based Event Flow

The payment and order lifecycle is managed through events as follows:
Event Flow Description

- **Payment Initialized**
  Payment Service publishes a PaymentInitialized event.
  Indicates the start of the transaction lifecycle.

- **Order State Change**
  Order Service consumes the event and updates order state to PENDING.
  Publishes a stock reservation command.
  Stock Reservation
    - Inventory Service reserves stock for the given order.
    - Reservation is tied to the **Order ID** for safe retries.

  Payment Outcome
    - Payment Service publishes either PaymentConfirmed or PaymentFailed.

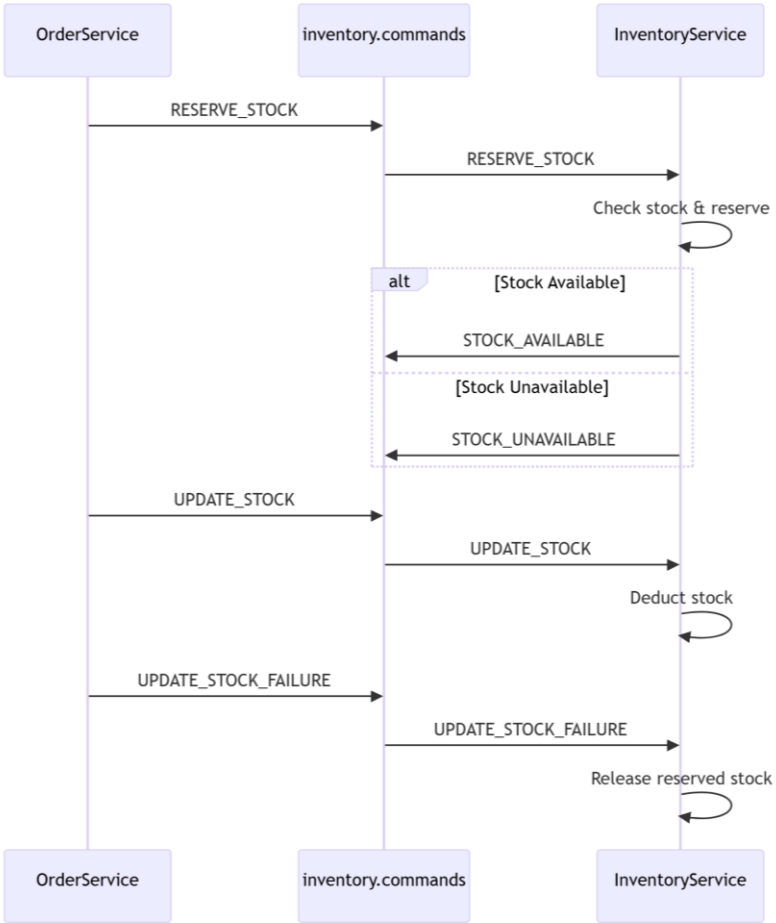
  Final Order and Inventory Update
    - Order Service updates order state to COMPLETED for both payment outcomes.
    - Inventory Service finalizes stock updates accordingly.

This flow ensures that all services progress independently while remaining synchronized through events.

5.3 Messaging Queues and Topics

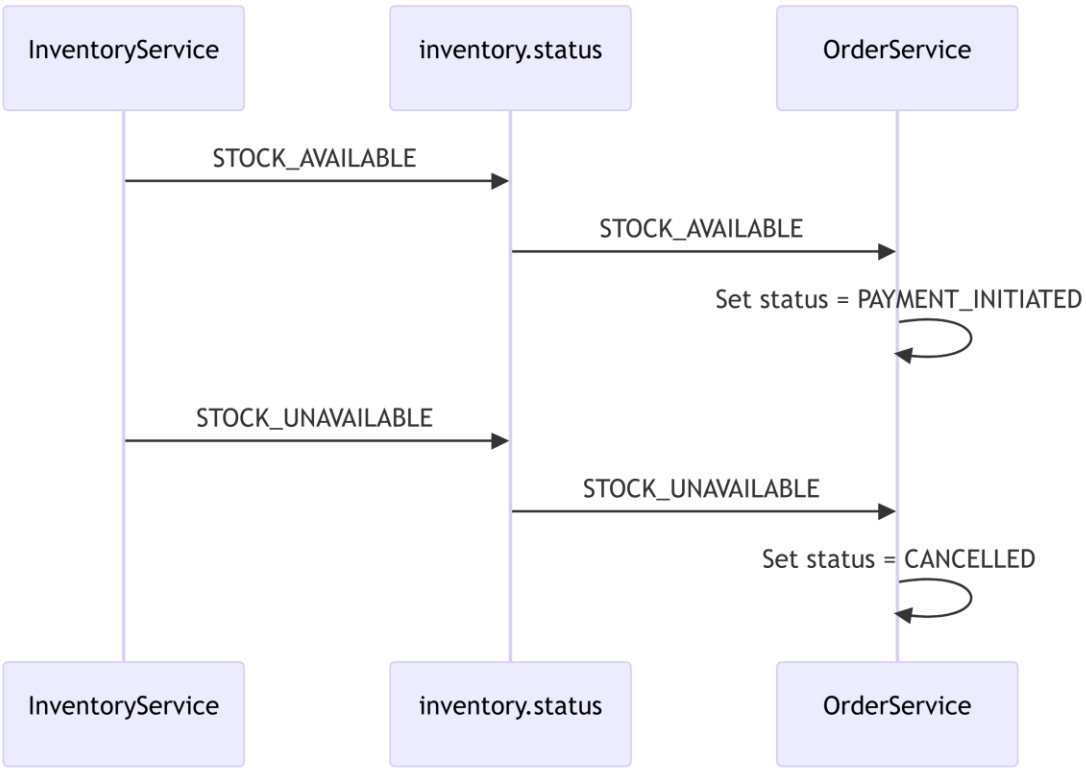
inventory.commands

- **Producer**: Order Service
- **Consumer:** Inventory Service
- **Purpose:** Sends product ID, quantity, and Order ID after order creation. Inventory Service attempts to reserve the requested stock.



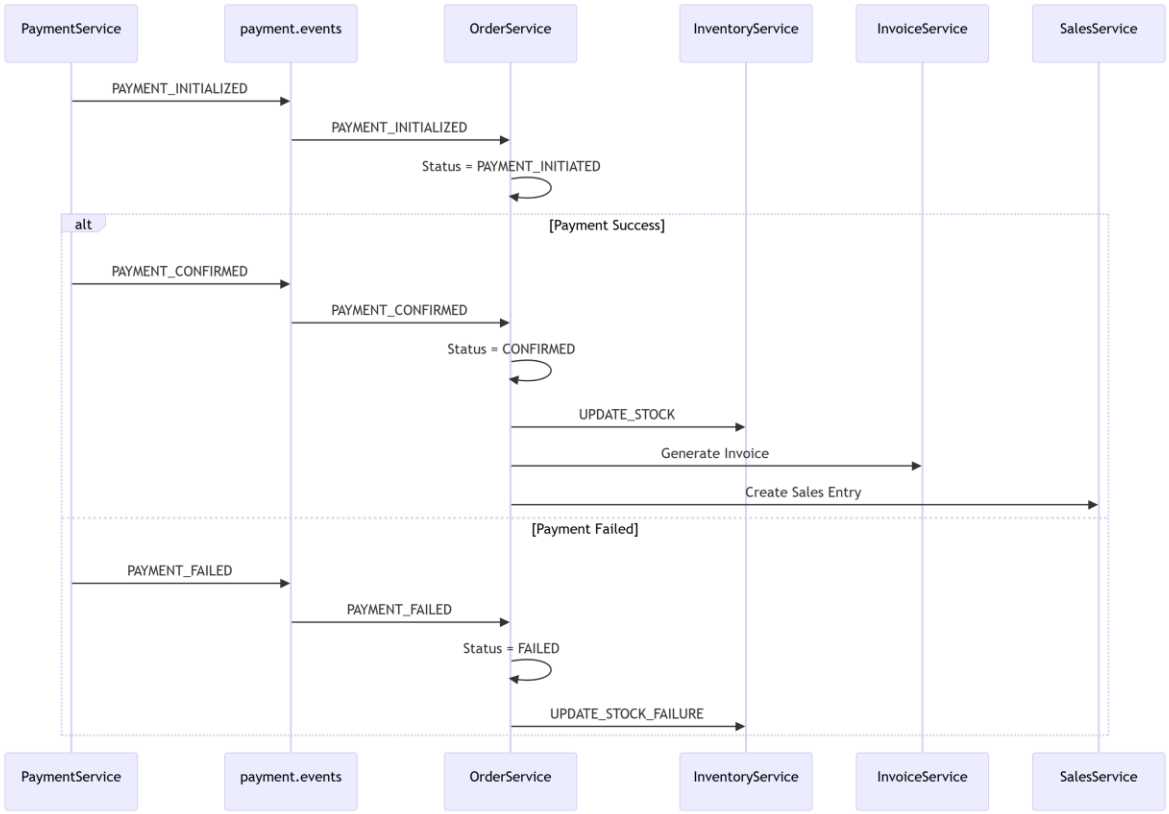
inventory.status

- **Producer**: Inventory Service
- **Consumer**: Order Service
- **Purpose**: Communicates with stock reservation status. If reservation fails, Order Service performs compensating logic and cancels the order.



payment.events

- **Producer**: Payment Service (via Stripe webhook integration)
- **Consumer**: Order Service
- **Purpose**: Publishes payment status events (PaymentInitialized, PaymentConfirmed, PaymentFailed) that drive order state transitions.



5.4 Idempotent Message Processing

To safely handle message retries and duplicate deliveries, InstaBuy enforces idempotency across all consumers.

Idempotency Key: Order ID

Implementation Strategy

- Every message includes the Order ID
- Services maintain a record of processed Order IDs
- If a message with an already-processed Order ID is received, it is safely ignored

Benefits

- Prevents duplicate stock reservations
- Prevents repeated order state transitions
- Enables safe retries without data corruption

Idempotent consumers are essential for reliable event processing in distributed systems where **at-least-once delivery** is expected.

5.5 Failure Handling and Dead Letter Queues (DLQ)

Automatic Retries

- ActiveMQ automatically retries message delivery upon transient failures
- Consumers can temporarily fail without losing messages
- Retries ensure resilience against network or service outages

Dead Letter Queues (DLQ)

- When a message cannot be processed after the configured retry attempts, it is routed to a **Dead Letter Queue (DLQ)**.

DLQ Handling Strategy

- No automatic reprocessing
- Messages are stored for investigation
- Used strictly for failure isolation

Audit Logs for DLQ Messages

Instead of re-driving messages automatically, InstaBuy maintains detailed audit logs for all DLQ entries:

- Order ID
- Event type
- Failure reason
- Timestamp
- Original message payload

Benefits

- Operational transparency
- Easier root cause analysis
- Compliance and traceability
- Prevents poison messages from blocking the system

5.6 Resilience and Availability Benefits

By combining asynchronous messaging with retries, idempotency, and DLQ-based auditing, InstaBuy achieves:

- Non-blocking communication under high traffic
- Independent service failure without cascading impact
- Eventual consistency without distributed locks
- Safe recovery from transient and permanent failures
- High system availability and scalability

6. Role-Based Access Control (RBAC)

Security is enforced at both the **API Gateway** and **service levels**, using JWT claims and Spring Security.

User

- Browse products
- Place orders
- Manage cart and checkout

Vendor

- Add and manage product listings
- View sales analytics via Recharts
- Restock inventory

Admin

- Perform global inventory operations
- Remove malicious or invalid product listings

RBAC ensures the principle of least privilege and prevents unauthorized access to sensitive operations.

7.Supporting Services and Additional Messaging Topics

To further enhance scalability, resilience, and operational reliability, InstaBuy extends its ActiveMQ-based asynchronous architecture beyond the core Saga flow. Several supporting background and utility services operate using event-driven communication and scheduled processing. These services remain loosely coupled and do not block the core order workflow.

7.1. OTP Cleanup Service

Purpose: Periodically cleans up expired, unused, or already processed OTP records to ensure security and database hygiene.

Behavior:

- Run every 10 minutes (scheduled job).

Identifies:

- Expired OTPs
- Inactive OTPs
- OTPs already marked as processed
- Deletes or archives these records from the OTP store.

Benefit:

- Prevents OTP reuse
- Reduces unnecessary data growth
- Improves authentication security

7.2. Invoice Generation Service

Purpose: Generates and stores digital invoices after successful payment.

Workflow:

- Stripe confirms payment via webhook

- Payment Service publishes a payment success event

Order Service:

- Aggregates order details
- Fetches buyer information
- Includes transaction/Order ID
- Digital invoice is generated (PDF)
- Invoice is stored and made available for user download

Benefit:

- Ensures invoices are generated only after confirmed payment
- Avoids synchronous invoice creation delays
- Supports regulatory and accounting requirements

7.3. Order Cleanup Service (Timeout Handling)

Purpose: Ensures system consistency when payment is initiated but not completed within a defined time window.

Behavior:

- Monitors orders stuck in PENDING state
- If an order remains unconfirmed for more than 10 minutes:
- Marks order as FAILED
- Emits an event to unreserve previously allocated stock

Benefit:

- Prevents inventory lock-ups
- Handles abandoned or failed payments automatically
- Improves inventory accuracy

7.4. Failed Payment Recovery & Admin Reconciliation Service

Purpose: Handles edge cases where payment succeeds, but the corresponding order is missing or corrupted.

Scenario:

- Payment confirmed successfully
- Order record not found or processing failed

Behavior:

- Stores payment transaction details in a separate failure datastore
- Flags the transaction for manual admin review
- Enables administrators to:
- Reconcile data inconsistencies
- Trigger refunds back to users if required

Benefit:

- Prevents silent payment losses

- Ensures customer trust
- Adds financial auditability and rollback capability

8. Conclusion

The InstaBuy platform exemplifies a production-grade microservices architecture designed to operate reliably under high load while maintaining consistency, security, and scalability. By adopting **Netflix Eureka for dynamic service discovery**, the system enables services to scale independently and remain resilient during failures or redeployments.

A **choreography-based Saga pattern**, implemented through **event-driven communication**, ensures data consistency across distributed services without relying on blocking synchronous calls or two-phase commits. This design significantly improves fault tolerance and allows each service to manage its own local transactions while responding to domain events. **ActiveMQ** serves as a dependable messaging backbone, enabling asynchronous communication, retry mechanisms, and decoupled service interaction—critical for maintaining responsiveness in a distributed environment.

The inclusion of **supporting background services**—such as OTP cleanup, order timeout handling, invoice generation, mailing, and payment anomaly recovery—further strengthens operational stability. These services address real-world failure scenarios, including abandoned payments, missing orders after successful transactions, and stale inventory reservations, ensuring system reliability and financial integrity.

Security is enforced through **Role-Based Access Control (RBAC)**, protecting sensitive endpoints and ensuring that users, administrators, and internal services operate within well-defined authorization boundaries. Combined with external payment validation via **Stripe webhooks**, InstaBuy achieves a secure and auditable transaction lifecycle.

Overall, the platform demonstrates how modern microservices principles—**loose coupling, eventual consistency, asynchronous messaging, and fault isolation**—can be effectively applied to build a scalable, resilient, and extensible e-commerce system capable of supporting large-scale concurrent users.

9.Future Improvements and Enhancements

Event Streaming and Scalability

Evaluate migration from ActiveMQ to Apache Kafka for:

- Higher throughput

- Event replicability

Long-term event storage:

- This would be particularly useful for analytics, audit trails, and advanced monitoring.

Saga State Monitoring Dashboard

Implement a Saga tracking dashboard to visualize:

- Order states
- Compensation events
- Timeouts and failures

Helps operations teams quickly identify stuck or failed transactions.

Security Enhancements

- Add OAuth 2.0 for API authentication.
- Implement token-based service-to-service authentication.
- Add encryption for sensitive payloads in message queues.
- Multi-Factor Authentication (MFA) as an additional layer of protection for user and admin access

Automated DLQ Analysis and Alerting

While DLQs are currently supported through audit logs, future enhancements may add:

- Automated DLQ dashboards
- Intelligent failure classification
- Alerting when DLQ thresholds exceed limits

Optional controlled replay mechanisms after root cause fixes

- This reduces manual intervention and improves operational maturity.